Odpowiedzi na pytania z zakresu informatyki - Egzamin inżynierski

1. Indeksy w bazach danych

Indeksy rzadkie i gęste

- **Indeksy rzadkie** (sparse indexes):
  - Zawierają wpisy tylko dla **wybranych wierszy** w tabeli (często jeden na blok danych)
  - **Zalety**: zajmują mniej miejsca, wymagają mniej operacji aktualizacji
  - **Wady**: wolniejsze wyszukiwanie niż w indeksach gęstych
  - **Zastosowanie**: przydatne dla kolumn z powtarzającymi się wartościami lub zawierającymi NULL
- **Indeksy gęste** (dense indexes):
  - Zawierają wpisy dla **każdego wiersza** w tabeli
  - **Zalety**: szybsze wyszukiwanie, brak konieczności przeglądania bloków danych
  - **Wady**: większe zużycie pamięci, wyższe koszty operacji modyfikacji
  - **Zastosowanie**: idealne dla kolumn z **unikalnymi wartościami**

Indeksy unikalne

- **Wymuszają unikalność** wartości w indeksowanej kolumnie lub zestawie kolumn
- **Zapobiegają duplikatom** danych (np. adresy email, numery PESEL)
- **Wspierają klucze główne** i integralność danych
- **Implementacja**: `CREATE UNIQUE INDEX nazwa_indeksu ON tabela(kolumna);`
- **Zalety**: gwarantują integralność danych, przyśpieszają operacje wyszukiwania
- **Wady**: wyższe koszty operacji modyfikacji danych, większe zapotrzebowanie na zasoby

Indeksy proste

- Utworzone na **pojedynczej kolumnie** tabeli
- **Przyspieszają wyszukiwanie** na podstawie wartości w pojedynczej kolumnie
- **Wsparcie dla operacji sortowania** (ORDER BY) i filtrowania (WHERE)
- **Zalety**: prosta struktura, łatwość utrzymania, dobra wydajność dla zapytań na pojedynczej kolumnie
- **Wady**: ograniczona efektywność przy złożonych zapytaniach wykorzystujących wiele kolumn

B+ drzewa

- **Najczęściej stosowana struktura** do implementacji indeksów w bazach danych
- **Zrównoważone drzewo wyszukiwania** zoptymalizowane pod kątem dostępu do dysku

- **Właściwości**:
  - Wszystkie liście znajdują się na **tym samym poziomie**
  - **Węzły wewnętrzne** zawierają tylko klucze i wskaźniki do węzłów potomnych
  - **Węzły liściowe** przechowują wartości kluczy i wskaźniki do rekordów
  - Liście są **połączone w listę** umożliwiającą sekwencyjny dostęp
  - Każdy węzeł (poza korzeniem) musi być wypełniony przynajmniej w **połowie**
- **Zalety**:
  - **Optymalizacja dostępu do dysku** - minimalizacja operacji I/O
  - **Efektywność przy dużych zbiorach** danych
  - **Zrównoważona wysokość** - logarytmiczna złożoność wyszukiwania
  - **Wsparcie dla dostępu sekwencyjnego**

Rola indeksów

- **Przyspieszenie wyszukiwania** danych bez konieczności przeglądania całej tabeli
- **Poprawa wydajności złożonych zapytań** (WHERE, JOIN, ORDER BY, GROUP BY)
- **Zmniejszenie kosztów operacji we/wy** (I/O) - ograniczenie liczby odczytów z dysku
- **Wsparcie dla kluczy głównych i obcych** - utrzymanie integralności referencyjnej
- **Zapewnienie lepszej skalowalności** aplikacji przy rosnącym wolumenie danych

Koszty zastosowania indeksów

- **Zwiększone zużycie przestrzeni dyskowej** - każdy indeks wymaga dodatkowego miejsca
- **Obciążenie pamięci operacyjnej** - indeksy są buforowane podczas przetwarzania zapytań
- **Narzut na operacje modyfikacji danych**:
  - Spowolnienie operacji **INSERT** (aktualizacja wszystkich indeksów)
  - Obciążenie przy operacjach **UPDATE** (szczególnie przy zmianach w kolumnach indeksowanych)
  - Opóźnienia w operacjach **DELETE** (aktualizacja struktur indeksów)
- **Blokady dostępu** - potencjalne problemy z równoczesnym dostępem podczas aktualizacji indeksów
- **Optymalizacja kosztów**:
  - Indeksowanie tylko **często używanych** kolumn
  - Unikanie **nadmiernej liczby indeksów** na tabelach z częstymi zmianami
  - **Okresowa reorganizacja** i defragmentacja indeksów

2. Kontrola dostępu w relacyjnych bazach danych

Kontrola dostępu do bazy

- **Metoda uznaniowa** (Discretionary Access Control - **DAC**):
  - **Najpowszechniej stosowana** metoda w systemach relacyjnych
  - **Indywidualne określenie uprawnień** dla każdego użytkownika i obiektu
  - **Administrator/właściciel** obiektu może przyznawać i odbierać prawa dostępu
  - **Standardowe uprawnienia**: SELECT, INSERT, UPDATE, DELETE, REFERENCES
  - **Zaimplementowana** w standardzie SQL92
- **Metoda obligatoryjna** (Mandatory Access Control - **MAC**):
  - **Bardziej rygorystyczna** metoda stosowana w systemach wymagających wysokiego bezpieczeństwa
  - **Każdy obiekt i użytkownik** posiada określoną klasę poufności
  - **Model Bell-LaPadula** z poziomami: ściśle tajne, tajne, poufne, bez klauzuli
  - **Zasady dostępu**:
    - **Prawo odczytu** dla użytkowników o klasie wyższej/równej klasie obiektu
    - **Prawo zapisu** dla użytkowników o klasie równej/niższej od klasy obiektu

Kontrola dostępu do schematu i obiektów schematu

- **Schemat** to logiczna jednostka organizacyjna grupująca powiązane obiekty
- **Kontrola dostępu na poziomie schematu** pozwala na przydzielanie uprawnień do całych grup obiektów
- **Typy obiektów schematu** podlegające kontroli:
  - **Tabele** zawierające dane w postaci wierszy i kolumn
  - **Widoki (perspektywy)** - wirtualne tabele oparte na zapytaniach SQL
  - **Procedury składowane** - bloki kodu SQL przechowywane w bazie
  - **Funkcje** - podobne do procedur, ale zwracające wartość
  - **Sekwencje** - generatory unikalnych wartości
  - **Typy danych** zdefiniowane przez użytkownika
- **Perspektywy (widoki)** jako mechanizm kontroli dostępu:
  - **Ograniczenie dostępu** do określonych kolumn lub wierszy tabeli
  - **Różne poziomy dostępu** dla różnych użytkowników
  - **Ukrycie złożoności** rzeczywistej struktury danych
  - **Zwiększenie bezpieczeństwa** przez ukrycie bezpośredniego dostępu
  - **Uproszczenie złożonych zapytań** dla użytkowników końcowych

```sql
-- Przykład wykorzystania widoku do kontroli dostępu
CREATE VIEW bezpieczne_dane AS
SELECT imie, nazwisko, departament
FROM pracownicy
WHERE lokalizacja = 'Warszawa';

GRANT SELECT ON bezpieczne_dane TO rola_analityk;
```

Użytkownicy i role w relacyjnych bazach danych

- **Użytkownik** to podmiot (osoba, aplikacja, system) łączący się z bazą danych:
  - **Unikalna nazwa** identyfikująca
  - **Mechanizm uwierzytelniania** (najczęściej hasło)
  - **Zestaw uprawnień** określających dozwolone operacje
- **Role** jako mechanizm grupowania uprawnień:
  - **Zestawy uprawnień** przypisywane wielu użytkownikom
  - **Uproszczenie administracji** poprzez zarządzanie grupami użytkowników
  - **Odzwierciedlenie struktur organizacyjnych** i funkcji zawodowych
  - **Łatwiejsze zarządzanie zmianami** w uprawnieniach
  - **Zwiększenie bezpieczeństwa** poprzez standaryzację dostępu

```sql
-- Przykład tworzenia i zarządzania rolami
CREATE ROLE analityk_danych;
GRANT SELECT ON schema_finanse.* TO analityk_danych;
GRANT analityk_danych TO 'maria'@'localhost';
```

Role-Based Access Control (RBAC)

- **Model kontroli dostępu** oparty na rolach - obecnie standard w zarządzaniu uprawnieniami
- **Komponenty RBAC**:
  - **Użytkownicy**: podmioty potrzebujące dostępu do danych
  - **Role**: zestawy uprawnień odpowiadające funkcjom zawodowym
  - **Uprawnienia**: określone operacje na obiektach bazy danych
- **Zasada działania RBAC**:
  - Użytkownikom **przypisuje się role** odpowiadające ich funkcjom
  - Role **otrzymują uprawnienia** niezbędne do określonych zadań
  - Użytkownik **dziedziczy wszystkie uprawnienia** przypisanych ról

    - Zmiana uprawnień roli **automatycznie aktualizuje** uprawnienia użytkowników
- **Typowe role** w organizacji:
    - **Administrator bazy danych (DBA)**: pełny dostęp do wszystkich obiektów
    - **Deweloper**: uprawnienia do tworzenia i modyfikowania obiektów
    - **Analityk danych**: dostęp do odczytu danych, raportowania
    - **Operator aplikacji**: ograniczone uprawnienia do typowych operacji
    - **Audytor**: dostęp tylko do odczytu i przeglądania logów
- **Korzyści z RBAC**:
    - **Uproszczona administracja** na poziomie grup
    - **Zasada najmniejszych uprawnień** - użytkownicy otrzymują minimum
    - **Skalowalność** - łatwe dostosowanie do zmian organizacyjnych
    - **Standaryzacja** uprawnień dla podobnych funkcji
    - **Zwiększenie bezpieczeństwa** - redukcja ryzyka nadmiernych uprawnień

3. Protokół HTTP

Struktura żądania i odpowiedzi HTTP

Struktura żądania HTTP

- **Linia żądania**: `metoda URI wersja` (np. `GET /index.html HTTP/1.1`)
- **Nagłówki żądania**: informacje o kliencie, preferencje
- **Pusta linia** (CRLF) oddzielająca nagłówki od treści
- **Treść żądania** (opcjonalna, np. w przypadku POST)

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html
[pusta linia]
[treść żądania - opcjonalna]
```

Struktura odpowiedzi HTTP

- **Linia statusu**: `wersja kod_statusu opis` (np. `HTTP/1.1 200 OK`)
- **Nagłówki odpowiedzi**: informacje o serwerze, zawartości
- **Pusta linia** (CRLF)
- **Treść odpowiedzi** (body)

```
HTTP/1.1 200 OK
Date: Mon, 23 May 2022 22:38:34 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 138
[pusta linia]
<!DOCTYPE html>
<html>
<head><title>Przykład</title></head>
<body><h1>Witaj, świecie!</h1></body>
</html>
```

Typowe nagłówki HTTP

- **Nagłówki żądania**:
  - **Host**: określa docelowy host i port
  - **User-Agent**: identyfikuje przeglądarkę/klienta
  - **Accept**: określa akceptowane typy MIME
  - **Accept-Language**: preferencje językowe klienta
  - **Authorization**: dane uwierzytelniające
  - **Cookie**: przesyła ciasteczka przechowywane po stronie klienta
  - **Referer**: adres URL strony, z której klient przyszedł
- **Nagłówki odpowiedzi**:
  - **Server**: informacje o serwerze HTTP
  - **Content-Type**: typ MIME zwracanej zawartości
  - **Content-Length**: rozmiar treści w bajtach
  - **Set-Cookie**: ustawia ciasteczka po stronie klienta
  - **Cache-Control**: dyrektywy dotyczące buforowania
  - **ETag**: identyfikator wersji zasobu
  - **Location**: używany przy przekierowaniach

GET vs POST

Metoda GET

- **Przeznaczenie**: pobieranie zasobów
- **Parametry**: przesyłane w URL (query string)
- **Widoczność**: parametry widoczne w pasku adresu
- **Ograniczenia**: limit długości URL (zwykle ok. 2000 znaków)
- **Idempotentność**: wielokrotne wywołanie daje ten sam efekt

- **Buforowanie**: może być buforowane przez przeglądarki i proxy
- **Zakładki**: można zapisać jako zakładkę z parametrami
- **Przykład**: `GET /search?q=przykład&lang=pl HTTP/1.1`

Metoda POST

- **Przeznaczenie**: przesyłanie danych do przetworzenia
- **Parametry**: przesyłane w ciele żądania
- **Widoczność**: parametry nie są widoczne w URL
- **Ograniczenia**: praktycznie bez limitu rozmiaru danych
- **Idempotentność**: powtórne wywołanie może dawać inny efekt
- **Buforowanie**: zwykle nie jest buforowane
- **Zakładki**: nie można zapisać parametrów w zakładce
- **Bezpieczeństwo**: lepsze dla danych poufnych (ale wciąż wymagane HTTPS)

Inne metody HTTP

- **PUT**: aktualizacja/zastąpienie istniejącego zasobu
- **DELETE**: usunięcie zasobu
- **HEAD**: jak GET, ale zwraca tylko nagłówki bez treści
- **OPTIONS**: informacja o dostępnych metodach dla zasobu
- **PATCH**: częściowa aktualizacja zasobu
- **TRACE**: diagnostyczna metoda odbijająca żądanie do klienta
- **CONNECT**: ustanawia tunel do serwera przez proxy

Bezstanowość HTTP

- Protokół HTTP jest **bezstanowy** - każde żądanie jest **niezależne i autonomiczne**
- Serwer **nie przechowuje informacji** o poprzednich żądaniach klienta
- **Każde żądanie** musi zawierać wszystkie informacje potrzebne do jego przetworzenia
- **Techniki emulacji stanu**:
  - **Ciasteczka** (cookies) przechowywane po stronie klienta
  - **Tokeny sesji** przesyłane w żądaniach
  - **Parametry URL** przechowujące identyfikatory sesji
  - **Ukryte pola formularzy** z danymi sesji
  - **Web Storage API** (localStorage, sessionStorage)
- **Zalety bezstanowości**:

    - **Prostota** implementacji serwera
    - **Skalowalność** - łatwiejsze równoważenie obciążenia
    - **Odporność na awarie** - każde żądanie jest samowystarczalne
- **Wady bezstanowości**:
    - **Konieczność ponownego przesyłania** kontekstu w każdym żądaniu
    - **Większy ruch sieciowy** przy złożonych sesjach
    - **Złożoność implementacji** aplikacji wymagających stanu

4. Transakcje w bazach danych

Rola transakcji

- **Logiczna jednostka pracy** w bazie danych
- **Grupowanie operacji**, które muszą być wykonane jako całość
- **Ochrona integralności danych** podczas współbieżnego dostępu
- **Mechanizm odzyskiwania** po awariach
- **Izolacja współbieżnych operacji** wielu użytkowników
- **Przykład**: przelew bankowy (odjęcie z jednego konta, dodanie do drugiego)

Własności ACID

- **Atomowość** (Atomicity):
    - Transakcja jest **niepodzielną jednostką pracy**
    - Albo **wszystkie operacje** transakcji się wykonują
    - Albo **żadna** (w przypadku błędu następuje rollback)
- **Spójność** (Consistency):
    - Transakcja przekształca bazę z **jednego spójnego stanu w drugi**
    - **Wszystkie reguły integralności** są spełnione po zakończeniu
    - **Ograniczenia, wyzwalacze, więzy integralności** są respektowane
- **Izolacja** (Isolation):
    - Transakcje są **izolowane od siebie nawzajem**
    - Efekty niedokończonej transakcji **nie są widoczne** dla innych
    - Różne **poziomy izolacji** oferują różne gwarancje
- **Trwałość** (Durability):
    - Po zatwierdzeniu (commit), zmiany są **trwale zapisane**
    - **Awaria systemu** nie spowoduje utraty zatwierdzonych zmian
    - Realizowane przez **dzienniki transakcyjne** i inne mechanizmy

Poziomy izolacji ANSI

- **Read Uncommitted** (poziom 0):
  - **Najniższy poziom** izolacji
  - Transakcje widzą **niezatwierdzone zmiany** innych transakcji
  - Możliwy **brudny odczyt** (dirty read)
  - **Maksymalna współbieżność**, minimalna izolacja
- **Read Committed** (poziom 1):
  - Transakcje widzą tylko **zatwierdzone zmiany** innych transakcji
  - Eliminuje **brudny odczyt**
  - Możliwe **niepowtarzalne odczyty** (non-repeatable read)
  - **Standardowy poziom** w wielu bazach danych
- **Repeatable Read** (poziom 2):
  - Gwarantuje, że **odczyty się powtarzają** - te same zapytania dają te same wyniki
  - Eliminuje **niepowtarzalne odczyty**
  - Możliwe **fantomy** (phantom reads) - nowe wiersze spełniające warunek zapytania
  - **Blokowanie odczytanych wierszy** do końca transakcji
- **Serializable** (poziom 3):
  - **Najwyższy poziom** izolacji
  - Transakcje wykonywane są tak, jakby działały **sekwencyjnie**
  - Eliminuje **wszystkie anomalie** współbieżności
  - **Najniższa współbieżność**, maksymalna izolacja
  - Realizowane przez **blokowanie predykatowe** lub inne mechanizmy

Dzienniki transakcyjne

- **Rejestry wszystkich zmian** wprowadzanych przez transakcje
- **Kluczowy mechanizm** dla zapewnienia trwałości i odzyskiwania
- **Typy dzienników**:
  - **Redo log** - zapisuje informacje potrzebne do ponownego wykonania operacji
  - **Undo log** - zapisuje informacje potrzebne do cofnięcia operacji
  - **Dzienniki fizyczne** - zapisują zmiany na poziomie stron/bloków danych
  - **Dzienniki logiczne** - zapisują operacje na poziomie semantycznym
- **Działanie**:
  - **Write-Ahead Logging (WAL)** - zmiany najpierw zapisywane do dziennika, potem do bazy

- **Checkpointing** - okresowe zapisywanie punktów synchronizacji
- **Recovery** - odtwarzanie stanu po awarii z wykorzystaniem dzienników

Rola blokad

- **Mechanizm synchronizacji** zapewniający izolację transakcji
- **Typy blokad**:
  - **Blokady współdzielone (S, shared)** - dla operacji odczytu
  - **Blokady wyłączne (X, exclusive)** - dla operacji zapisu
  - **Blokady intencji** - sygnalizują zamiar blokowania na niższym poziomie
- **Granularność blokad**:
  - **Blokowanie na poziomie bazy danych**
  - **Blokowanie na poziomie tabeli**
  - **Blokowanie na poziomie strony**
  - **Blokowanie na poziomie wiersza**
  - **Blokowanie na poziomie pola**
- **Problemy blokowania**:
  - **Zakleszczenie (deadlock)** - transakcje blokują się nawzajem
  - **Zagłodzenie (starvation)** - transakcja nie może uzyskać blokady
  - **Konwoje blokad** - transakcje czekają w kolejce
- **Strategie obsługi zakleszczenia**:
  - **Wykrywanie i przerywanie** - przerwanie jednej z transakcji
  - **Zapobieganie** - porządkowanie żądań blokad
  - **Unikanie** - przydzielanie blokad tylko gdy bezpieczne

Wielowersjowa kontrola współbieżności (MVCC)

- **Alternatywa dla blokad**, zwiększająca współbieżność
- **Zasada działania**:
  - **Każda modyfikacja** tworzy nową wersję danych
  - **Każda transakcja** widzi spójny stan bazy z momentu rozpoczęcia
  - **Stare wersje** są zachowywane tak długo, jak są potrzebne
  - **Konflikty** rozwiązywane przez odrzucenie niektórych transakcji
- **Zalety MVCC**:
  - **Czytający nie blokują piszących** i odwrotnie
  - **Wysoka współbieżność** - lepsze wykorzystanie zasobów

    - **Spójny widok danych** dla długotrwałych transakcji
- **Implementacje**:
    - **PostgreSQL**: każdy wiersz ma dodatkowe kolumny metadanych
    - **Oracle**: używa segmentów undo
    - **MySQL (InnoDB)**: wykorzystuje dziennik undo
- **Wyzwania**:
    - **Zarządzanie wersjami** - usuwanie niepotrzebnych wersji
    - **Zwiększone zużycie pamięci** do przechowywania wielu wersji
    - **Złożoność implementacji** i rozwiązywania konfliktów

5. Wybrane algorytmy klasyfikacji

Algorytm naiwny Bayesa

- **Definicja**: Probabilistyczny klasyfikator bazujący na **twierdzeniu Bayesa**
- **"Naiwny"** ponieważ zakłada **niezależność cech** (atrybutów)
- **Wzór Bayesa**: $P(Y|X) = [P(X|Y) \times P(Y)] / P(X)$
    - $P(Y|X)$: prawdopodobieństwo klasy Y przy danych cechach X
    - $P(X|Y)$: prawdopodobieństwo cech X w klasie Y
    - $P(Y)$: prawdopodobieństwo a priori klasy Y
    - $P(X)$: prawdopodobieństwo wystąpienia cech X
- **Klasyfikacja**: wybiera klasę z najwyższym prawdopodobieństwem
    - $y = \text{argmax } P(Y=y) \times \prod P(X_i|Y=y)$
- **Warianty naiwnego Bayesa**:
    - **Gaussian NB**: dla cech o rozkładzie normalnym
    - **Multinomial NB**: dla cech dyskretnych (np. zliczanie słów)
    - **Bernoulli NB**: dla binarnych cech (występuje/nie występuje)
- **Zalety**:
    - **Prosty i szybki** algorytm
    - **Wydajny obliczeniowo** - liniowa złożoność
    - **Dobry dla wysokowymiarowych danych** (np. klasyfikacja tekstu)
    - **Radzi sobie z małymi zbiorami treningowymi**
    - **Niewrażliwy na nieistotne cechy**
- **Wady**:
    - **Założenie niezależności** cech często nierealistyczne

    - **Problemy z cechami nieobecnymi** w zbiorze treningowym
    - **Wrażliwy na cechy skorelowane**
    - Może dawać **słabe oszacowanie prawdopodobieństw**
- **Zastosowania**:
    - **Klasyfikacja tekstu** i analiza sentymentu
    - **Filtrowanie spamu**
    - **Diagnostyka medyczna**
    - **Systemy rekomendacji**

k-NN (k-Najbliższych Sąsiadów)

- **Definicja**: Klasyfikator przypisujący obiekt do klasy na podstawie **k najbliższych sąsiadów**
- **Nieparametryczny** algorytm (nie zakłada modelu danych)
- **Algorytm**:
    1. Dla nowego punktu **oblicz odległość** do wszystkich punktów treningowych
    2. **Wybierz k najbliższych sąsiadów**
    3. **Przypisz klasę** poprzez głosowanie (najczęstsza klasa wśród sąsiadów)
- **Parametry**:
    - **Liczba sąsiadów k**: zazwyczaj nieparzysta (unika remisów)
    - **Metryka odległości**:
        - **Euklidesowa**: $\sqrt{(\Sigma(xi-yi)^2)}$ - standardowa
        - **Manhattan**: $\Sigma|xi-yi|$ - dla przestrzeni miejskiej
        - **Minkowskiego**: $(\Sigma|xi-yi|^p)^{(1/p)}$ - uogólnienie
        - **Hamminga**: liczba różnic - dla danych kategorycznych
- **Warianty**:
    - **Ważone k-NN**: głosy ważone odwrotnością odległości
    - **kd-drzewa**: struktury danych przyspieszające wyszukiwanie najbliższych sąsiadów
- **Zalety**:
    - **Prosty, intuicyjny** algorytm
    - **Brak fazy uczenia** (lazy learner)
    - **Brak założeń** o rozkładzie danych
    - **Efektywny dla wielu zadań** klasyfikacji
    - **Naturalnie obsługuje** problemy wieloklasowe
- **Wady**:
    - **Obliczeniowo kosztowny** dla dużych zbiorów

    - **Wrażliwy na nieistotne cechy** i skalowanie
    - **Problem z doborem optymalnego k**
    - **Wrażliwy na niezbalansowane** dane
    - **Wymaga przechowywania** całego zbioru treningowego
- **Zastosowania**:
    - **Rozpoznawanie wzorców**
    - **Systemy rekomendacji**
    - **Analiza anomalii**
    - **Wykrywanie oszustw**

Regresja logistyczna

- **Definicja**: Pomimo nazwy, to algorytm **klasyfikacji** (nie regresji)
- **Modeluje prawdopodobieństwo** przynależności do klasy za pomocą **funkcji logistycznej**
- **Funkcja logistyczna (sigmoid)**: $P(y=1|x) = 1/(1+e^{(-z)})$, gdzie $z = w_0 + w_1x_1 + ... + w_nx_n$
- **Granica decyzyjna**: liniowa w przestrzeni cech
- **Uczenie**:
    - **Maksymalizacja wiarygodności** (maximum likelihood)
    - **Minimalizacja funkcji kosztu** (cross-entropy loss)
    - **Gradient descent** do znalezienia optymalnych wag
- **Warianty**:
    - **Binarna regresja logistyczna**: dwie klasy
    - **Wieloklasowa regresja logistyczna**:
        - **One-vs-Rest (OvR)**: n klasyfikatorów binarnych
        - **Softmax**: uogólnienie funkcji logistycznej na wiele klas
- **Zalety**:
    - **Efektywna obliczeniowo**
    - **Probabilistyczna interpretacja** wyników
    - **Łatwa interpretacja** wag modelu (wpływ cech)
    - **Dobrze działa** z dużymi zbiorami danych
    - **Odporna na overfitting** (w porównaniu do bardziej złożonych modeli)
- **Wady**:
    - **Zakłada liniową granicę decyzyjną**
    - **Wrażliwa na współliniowość** cech
    - **Problemy z silnie niezbalansowanymi** danymi

    - **Wymaga normalizacji** cech
    - **Ograniczona ekspresyjność** dla złożonych zależności
- **Zastosowania**:
    - **Medycyna**: diagnoza chorób
    - **Marketing**: przewidywanie konwersji
    - **Finanse**: ocena ryzyka kredytowego
    - **Nauki społeczne**: modelowanie zachowań

6. Wzorzec MVC w wybranym frameworku webowym

Pojęcie wzorca MVC

- **Definicja**: Architektoniczny wzorzec projektowy rozdzielający aplikację na trzy komponenty:
    - **Model**: reprezentacja danych i logika biznesowa
    - **Widok**: prezentacja danych użytkownikowi (interfejs)
    - **Kontroler**: pośrednik między Modelem i Widokiem, obsługuje interakcje
- **Cel**: **Separacja odpowiedzialności** (Separation of Concerns)
    - **Oddzielenie prezentacji** od logiki biznesowej
    - **Większa modularność** i czytelność kodu
    - **Łatwiejsze utrzymanie** i rozbudowa
    - **Lepsze możliwości testowania** komponentów
    - **Równoległy rozwój** różnych części aplikacji
- **Przepływ danych w MVC**:
    1. Użytkownik wykonuje akcję w **Widoku**
    2. **Kontroler** odbiera akcję i przetwarza ją
    3. **Kontroler** aktualizuje **Model** w odpowiedzi na akcję
    4. **Model** powiadamia o zmianach (bezpośrednio lub pośrednio)
    5. **Widok** pobiera aktualne dane z **Modelu** i je prezentuje
    6. **Widok** oczekuje na kolejną akcję użytkownika

Implementacja MVC w Django

- **Django** - popularny framework webowy w języku Python
- **Django implementuje wariant MVC** nazywany **MTV** (Model-Template-View):
    - **Model w Django** = Model w MVC
    - **Template w Django** = Widok w MVC
    - **View w Django** = Kontroler w MVC

- **Model**:
  - Klasy Pythona dziedziczące po **django.db.models.Model**
  - **ORM** (Object-Relational Mapping) do interakcji z bazą danych
  - **Definicja struktury** danych i relacji
  - **Logika biznesowa** związana z danymi

```python
# Przykład modelu w Django
from django.db import models

class Author(models.Model):
    name = models.CharField(max_length=100)
    bio = models.TextField()

    def __str__(self):
        return self.name

class Book(models.Model):
    title = models.CharField(max_length=200)
    author = models.ForeignKey(Author, on_delete=models.CASCADE)
    publication_date = models.DateField()

    def __str__(self):
        return self.title
```

- **Template (Widok)**:
  - Szablony HTML z systemem szablonów Django
  - **Prezentacja danych** użytkownikowi
  - **Znaczniki szablonu** (template tags) do dynamicznego generowania treści
  - **Filtry** do formatowania danych

```html
<!-- Przykład szablonu w Django -->
{% extends "base.html" %}

{% block content %}
    <h1>{{ book.title }}</h1>
    <p>Autor: {{ book.author.name }}</p>
    <p>Data publikacji: {{ book.publication_date|date:"d.m.Y" }}</p>
{% endblock %}
```

- **View (Kontroler)**:
  - Funkcje lub klasy przetwarzające żądania HTTP

- **Obsługa interakcji** użytkownika
- **Komunikacja z Modelem**
- **Renderowanie szablonów** i przekazywanie danych

```python
# Przykład widoku funkcyjnego w Django
from django.shortcuts import render, get_object_or_404
from .models import Book

def book_detail(request, book_id):
    book = get_object_or_404(Book, id=book_id)
    return render(request, 'books/detail.html', {'book': book})

# Przykład widoku klasowego w Django
from django.views.generic import DetailView

class BookDetailView(DetailView):
    model = Book
    template_name = 'books/detail.html'
```

- **URLs (Routing)**:
  - **Mapowanie adresów URL** na widoki
  - **Przekazywanie parametrów** z URL do widoku

```python
# Przykład routingu w Django
from django.urls import path
from . import views

urlpatterns = [
    path('books/<int:book_id>/', views.book_detail, name='book_detail'),
    path('books/class/<int:pk>/', views.BookDetailView.as_view(), name='book_detail_class')
]
```

Mapowanie obiektowo-relacyjne

- **ORM** (Object-Relational Mapping) - technika konwersji między obiektami w kodzie a tabelami w bazie danych
- **Eliminuje potrzebę** pisania ręcznych zapytań SQL
- **Abstrakcja bazy danych** - uniezależnienie od konkretnego systemu bazodanowego
- **Model w MVC** często korzysta z ORM do interakcji z bazą danych
- **Komponenty ORM**:

    - **Definicja modelu**: klasy reprezentujące tabele
    - **Mapowanie typów**: konwersja między typami w języku programowania a typami w bazie
    - **Relacje**: odwzorowanie relacji między tabelami (one-to-one, one-to-many, many-to-many)
    - **Zarządzanie sesją**: śledzenie zmian obiektów
    - **Zapytania**: API do filtrowania, sortowania i pobierania danych
- **Implementacja w Django**:

  python

  ```python
  # Zapytania przy użyciu Django ORM
  # Pobieranie wszystkich książek
  books = Book.objects.all()

  # Filtrowanie
  recent_books = Book.objects.filter(publication_date__year=2023)

  # Relacje
  books_by_author = Book.objects.filter(author__name="J.K. Rowling")

  # Agregacje
  from django.db.models import Count
  author_stats = Author.objects.annotate(book_count=Count('book'))
  ```

- **Zalety ORM**:
    - **Szybszy rozwój** aplikacji
    - **Większa czytelność** kodu
    - **Bezpieczeństwo** (ochrona przed SQL Injection)
    - **Abstrakcja bazy danych**
    - **Automatyczne zarządzanie relacjami**
- **Wady ORM**:
    - **Spadek wydajności** dla złożonych zapytań
    - **Ukryta złożoność** generowanych zapytań SQL
    - **Krzywa uczenia się** dla zaawansowanych przypadków
    - **Ograniczenia** w wykorzystaniu specyficznych funkcji baz danych

Wzorce alternatywne

- **MVP (Model-View-Presenter)**:
    - **Presenter** pośredniczy między Modelem a Widokiem
    - **Większa separacja** Widoku od Modelu
    - **Widok jest pasywny** - tylko wyświetla dane

    - **Presenter zawiera logikę** prezentacji
    - **Łatwiejsze testowanie** dzięki większej izolacji komponentów
    - **Popularny w aplikacjach mobilnych** (Android)
- **MVVM (Model-View-ViewModel)**:
    - **ViewModel** konwertuje dane z Modelu na format dla Widoku
    - **Binding danych** (data binding) między Widokiem a ViewModelem
    - **Widok jest deklaratywny** - opisuje jak dane mają być wyświetlane
    - **Większa separacja odpowiedzialności**
    - **Popularny w aplikacjach frontend** (Angular, Vue.js, Knockout.js)
- **HMVC (Hierarchical MVC)**:
    - **Hierarchiczna organizacja** wielu triad MVC
    - **Lepsza modularność** dla złożonych aplikacji
    - **Każda triada** może pracować niezależnie
    - **Zwiększona skalowalność** i możliwość ponownego użycia kodu
    - **Popularny w dużych systemach**
- **MVC2 (Front Controller)**:
    - **Centralny kontroler** sterujący przepływem aplikacji
    - **Wszystkie żądania** przechodzą przez jeden punkt wejścia
    - **Rozdzielenie logiki nawigacji** od logiki biznesowej
    - **Uproszczona struktura** aplikacji
    - **Zastosowany w wielu frameworkach** (Laravel, Spring MVC)
- **PAC (Presentation-Abstraction-Control)**:
    - **Hierarchiczna struktura agentów**
    - Każdy agent składa się z komponentów **Prezentacji**, **Abstrakcji** i **Kontroli**
    - **Bardziej rozproszony** niż MVC
    - **Lepsze wsparcie** dla systemów rozproszonych
    - **Stosowany w złożonych interfejsach** użytkownika

Zalety i wady wzorca MVC

- **Zalety**:
    - **Rozdzielenie odpowiedzialności** (Separation of Concerns)
    - **Łatwiejsze testowanie** poszczególnych komponentów
    - **Równoległy rozwój** (różne zespoły mogą pracować nad M, V i C)
    - **Wielokrotne wykorzystanie kodu** (ten sam Model może być używany przez różne Widoki)

    - **Łatwiejsze utrzymanie i rozbudowa** aplikacji
- **Wady**:
    - **Zwiększona złożoność architektoniczna** dla prostych aplikacji
    - **Nadmiarowość** dla małych projektów
    - **Może powodować rozproszenie logiki biznesowej**
    - **Wymaga dobrego zrozumienia wzorca** przez zespół
    - **Trudności w określeniu granic** między komponentami